

BCEWithLogitsLoss

CLASS torch.nn.BCEWithLogitsLoss(*weight=None, size_average=None, reduce=None, reduction='mean', pos_weight=None*) [\[SOURCE\]](#)

This loss combines a *Sigmoid* layer and the *BCELoss* in one single class. This version is more numerically stable than using a plain *Sigmoid* followed by a *BCELoss* as, by combining the operations into one layer, we take advantage of the log-sum-exp trick for numerical stability.

The unreduced (i.e. with `reduction` set to `'none'`) loss can be described as:

$$\ell(x, y) = L = \{l_1, \dots, l_N\}^\top, \quad l_n = -w_n [y_n \cdot \log \sigma(x_n) + (1 - y_n) \cdot \log(1 - \sigma(x_n))],$$

where N is the batch size. If `reduction` is not `'none'` (default `'mean'`), then

$$\ell(x, y) = \begin{cases} \text{mean}(L), & \text{if reduction = 'mean'}; \\ \text{sum}(L), & \text{if reduction = 'sum'}. \end{cases}$$

This is used for measuring the error of a reconstruction in for example an auto-encoder. Note that the targets $t[i]$ should be numbers between 0 and 1.

It's possible to trade off recall and precision by adding weights to positive examples. In the case of multi-label classification the loss can be described as:

$$\ell_c(x, y) = L_c = \{l_{1,c}, \dots, l_{N,c}\}^\top, \quad l_{n,c} = -w_{n,c} [p_c y_{n,c} \cdot \log \sigma(x_{n,c}) + (1 - y_{n,c}) \cdot \log(1 - \sigma(x_{n,c}))],$$

where c is the class number ($c > 1$ for multi-label binary classification, $c = 1$ for single-label binary classification), n is the number of the sample in the batch and p_c is the weight of the positive answer for the class c .

$p_c > 1$ increases the recall, $p_c < 1$ increases the precision.

For example, if a dataset contains 100 positive and 300 negative examples of a single class, then `pos_weight` for the class should be equal to $\frac{300}{100} = 3$. The loss would act as if the dataset contains $3 \times 100 = 300$ positive examples.

Examples:

```
>>> target = torch.ones([10, 64], dtype=torch.float32) # 64 classes, batch size = 10
>>> output = torch.full([10, 64], 1.5) # A prediction (logit)
>>> pos_weight = torch.ones([64]) # All weights are equal to 1
>>> criterion = torch.nn.BCEWithLogitsLoss(pos_weight=pos_weight)
>>> criterion(output, target) # -log(sigmoid(1.5))
tensor(0.20...)
```

In the above example, the `pos_weight` tensor's elements correspond to the 64 distinct classes in a multi-label binary classification scenario. Each element in `pos_weight` is designed to adjust the loss function based on the imbalance between negative and positive samples for the respective class. This approach is useful in datasets with varying levels of class imbalance, ensuring that the loss calculation accurately accounts for the distribution in each class.

Parameters

- weight** (*Tensor, optional*) – a manual rescaling weight given to the loss of each batch element. If given, has to be a Tensor of size *nbatch*.
- size_average** (*bool, optional*) – Deprecated (see `reduction`). By default, the losses are averaged over each loss element in the batch. Note that for some losses, there are multiple elements per sample. If the field `size_average` is set to `False`, the losses are instead summed for each minibatch. Ignored when `reduce` is `False`. Default: `True`
- reduce** (*bool, optional*) – Deprecated (see `reduction`). By default, the losses are averaged or summed over observations for each minibatch depending on `size_average`. When `reduce` is `False`, returns a loss per batch element instead and ignores `size_average`. Default: `True`
- reduction** (*str, optional*) – Specifies the reduction to apply to the output: `'none'` | `'mean'` | `'sum'`. `'none'`: no reduction will be applied, `'mean'`: the sum of the output will be divided by the number of elements in the output, `'sum'`: the output will be summed. Note: `size_average` and `reduce` are in the process of being deprecated, and in the meantime, specifying either of those two args will override `reduction`. Default: `'mean'`
- pos_weight** (*Tensor, optional*) – a weight of positive examples to be broadcasted with target. Must be a tensor with equal size along the class dimension to the number of classes. Pay close attention to PyTorch's broadcasting semantics in order to achieve the desired operations. For a target of size `[B, C, H, W]` (where B is batch size) `pos_weight` of size `[B, C, H, W]` will apply different `pos_weights` to each element of the batch or `[C, H, W]` the same `pos_weights` across the batch. To apply the same positive weight along all spacial dimensions for a 2D multi-class target `[C, H, W]` use: `[C, 1, 1]`. Default: `None`

Shape:

- Input: $(*)$, where $*$ means any number of dimensions.
- Target: $(*)$, same shape as the input.
- Output: scalar. If `reduction` is `'none'`, then $(*)$, same shape as input.

Examples:

```
>>> loss = nn.BCEWithLogitsLoss()
>>> input = torch.randn(3, requires_grad=True)
>>> target = torch.empty(3).random_(2)
>>> output = loss(input, target)
>>> output.backward()
```

[◀ Previous](#)

[Next ▶](#)

© Copyright 2024, PyTorch Contributors.
Built with [Sphinx](#) using a [theme](#) provided by [Read the Docs](#).

Docs

Access comprehensive developer documentation for PyTorch
[View Docs](#)

Tutorials

Get in-depth tutorials for beginners and advanced developers
[View Tutorials](#)

Resources

Find development resources and get your questions answered
[View Resources](#)

PyTorch

Get Started

Features

Ecosystem

Blog

Contributing

Resources

Tutorials

Docs

Discuss

Github Issues

Brand Guidelines

Stay up to date

Facebook

Twitter

YouTube

LinkedIn

PyTorch Podcasts

Spotify

Apple

Google

Amazon

[Terms](#) | [Privacy](#)

© Copyright The Linux Foundation. The PyTorch Foundation is a project of The Linux Foundation. For web site terms of use, trademark policy and other policies applicable to The PyTorch Foundation please see www.linuxfoundation.org/policies/. The PyTorch Foundation supports the PyTorch open source project, which has been established as PyTorch Project a Series of LF Projects, LLC. For policies applicable to the PyTorch Project a Series of LF Projects, LLC, please see www.lfprojects.org/policies/.